

Trabajo Práctico

Trabajo Integrador Final – Organización Empresarial

Alumno

de la PEÑA, Juan Cruz

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Organización Empresarial

Docente Titular

VARBERDE, Francisco

Docente Tutor

VARBERDE, Francisco

19 de junio de 2026

Tabla de contenido

Trabajo Integrador Final – Organización Empresarial	1
Introducción	3
Objetivo General	3
Objetivos Específicos	3
Desarrollo	4
2.2 ARQUITECTURA Y PROGRAMACIÓN (El "Cómo")	4
2.2.1 Selección del Stack Tecnológico	4
2.2.2 Máquina de Estados	4
2.2.3 AS-IS (Proceso Actual - Manual/Sin Automatizar)	5
2.2.3 TO-BE (Proceso Optimizado)	5
2.2.3 Base de Datos Simulada	6
2.2.4 Lógica de Decisiones	6
2.3 Documentación y Calidad	7
2.3.1 Diccionario de Datos	7
2.3.2 Manual de Usuario	8
Manejo de Errores - BOT	9
Conclusión	10

Trabajo Práctico N°01

Introducción

Su finalidad es aplicar los conocimientos adquiridos a lo largo de la materia, integrando conceptos de modelado de procesos de negocio y desarrollo de software en un caso concreto.

Para este proyecto, se eligió el proceso de **gestión de solicitudes de soporte técnico** en una organización ficticia. Este proceso resulta adecuado porque presenta reglas de negocio claras, puntos de decisión definidos y una clara interacción entre el usuario y el sistema, lo que permite demostrar la utilidad de un chatbot como herramienta de automatización.

El trabajo se divide en dos etapas principales. En primer lugar, se realiza el modelado del proceso mediante la notación BPMN 2.0, distinguiendo el flujo actual ("as-is") del flujo optimizado con el bot ("to-be"). En segundo lugar, se implementa un chatbot funcional que ejecuta la lógica definida en el diagrama, incorporando gestión de estados, toma de decisiones y persistencia de datos.

El objetivo final es demostrar la coherencia entre el modelo de negocio y su implementación técnica, evidenciando que el futuro Técnico Universitario en Programación está capacitado para identificar ineficiencias operativas y proponer soluciones automatizadas en el ámbito empresarial.

Objetivo General

Automatizar el proceso de gestión de solicitudes de soporte técnico Nivel 1 mediante el desarrollo de un chatbot que permita clasificar, resolver o derivar incidentes tecnológicos, optimizando los tiempos de respuesta.

Objetivos Específicos

1. **Modelar el proceso de soporte técnico** (BPMN 2.0)
2. **Diseñar una máquina de estados**
3. **Implementar la lógica de clasificación y decisión** (categorías por problemas / soluciones)
4. **Desarrollar una base de datos simulada** (info de tickets generados)
5. **Incorporar mecanismos de robustez y manejo de errores** (permiten al bot gestionar entradas incorrectas)
6. **Documentar integralmente el proyecto**
7. **Presentar una solución funcional y documentada**

Desarrollo

2.2 ARQUITECTURA Y PROGRAMACIÓN (El "Cómo")

2.2.1 Selección del Stack Tecnológico

Para el desarrollo del chatbot se optó por el siguiente stack tecnológico:

- **Lenguaje de programación:** Python. Se eligió al ser el lenguaje utilizado en el total de la materia Programación I, por lo que será “cómoda” su implementación
- **Plataforma:** Whatsapp API, a través de la librería python-telegram-bot. Se seleccionó por su simplicidad de implementación y porque permite probar el bot de manera inmediata sin necesidad de configurar un servidor web completo.
- **Persistencia de datos:** Archivos JSON. Se utilizaron como base de datos simulada para almacenar tickets generados y la base de conocimiento con soluciones predefinidas. Esta elección permite simular la interacción con una base de datos real sin necesidad de configurar un motor de base de datos adicional.

2.2.2 Máquina de Estados

Los estados definidos para el proceso son:

Estado	Descripción
INICIO	El bot saluda al usuario y solicita que describa su problema.
ESPERANDO_DESCRIPCION	El bot espera que el usuario ingrese la descripción del problema.
CLASIFICANDO	El bot analiza el mensaje y clasifica la categoría del problema.
BUSCANDO_SOLUCION	El bot consulta la base de conocimiento en busca de una solución.
OFRECIENDO_SOLUCION	El bot muestra la solución encontrada y pregunta si funcionó.
CONFIRMANDO_SOLUCION	El bot espera la confirmación del usuario sobre la solución.
DERIVANDO	El bot deriva el caso a Nivel 2 y genera un ticket.
TICKET_CERRADO	El bot finaliza el proceso y cierra la conversación.

2.2.3 AS-IS (Proceso Actual - Manual/Sin Automatizar)

Descripción del proceso actual:

1. Inicio: El usuario tiene un problema técnico.
2. Acción: El usuario envía un correo electrónico o llama al departamento de soporte.
3. Tarea del operador: Un técnico de Nivel 1 recibe la solicitud manualmente.
4. Decisión del operador: El técnico lee el mensaje y decide qué categoría es (si sabe identificarlo).
5. Si es hardware: El técnico deriva manualmente el ticket a Nivel 2.
6. Si es software/acceso: El técnico busca en su memoria o en documentos la solución.
7. Si encuentra solución: El técnico responde por correo con la solución.
8. Si no encuentra solución: Deriva manualmente a Nivel 2.
9. Espera: El usuario prueba la solución y responde (si responde) por correo.
10. Confirmación: El técnico cierra el ticket manualmente si funcionó.
11. Si no funcionó: El técnico deriva nuevamente a Nivel 2.

Problemas del AS-IS:

- Tiempos de respuesta largos (depende de disponibilidad del técnico).
- Proceso manual y desorganizado (correos perdidos).
- Dependencia del conocimiento del técnico.
- No hay trazabilidad ni registro estructurado.
- Múltiples interacciones manuales.

2.2.3 TO-BE (Proceso Optimizado)

Descripción del proceso optimizado:

1. Inicio: El usuario escribe /start en Telegram.
2. Bot recibe mensaje: El bot saluda y solicita descripción del problema.
3. Usuario describe el problema: El usuario escribe el detalle.
4. Bot clasifica automáticamente: El bot analiza palabras clave y categoriza (hardware/acceso/software).
5. Si es hardware: El bot deriva automáticamente a Nivel 2 y genera ticket.
6. Si no es hardware: El bot busca en la base de conocimiento.
7. Si encuentra solución: El bot muestra la solución al usuario.
8. Si no encuentra solución: El bot deriva a Nivel 2 y genera ticket.
9. Bot pregunta confirmación: "¿Te funcionó? (Si/No)".
10. Si confirma: El bot cierra el ticket automáticamente.
11. Si no confirma: El bot deriva a Nivel 2 y genera ticket.

Ventajas del TO-BE:

- Respuesta inmediata: 24/7, sin esperar a un operador.
- Automatización: Clasifica, busca y deriva automáticamente.
- Trazabilidad: Todos los tickets quedan registrados en JSON.
- Base de conocimiento: Soluciones predefinidas accesibles al instante.
- Reducción de errores: No depende de la memoria del operador.
- Escalable: Puede atender múltiples usuarios simultáneamente.

2.2.3 Base de Datos Simulada

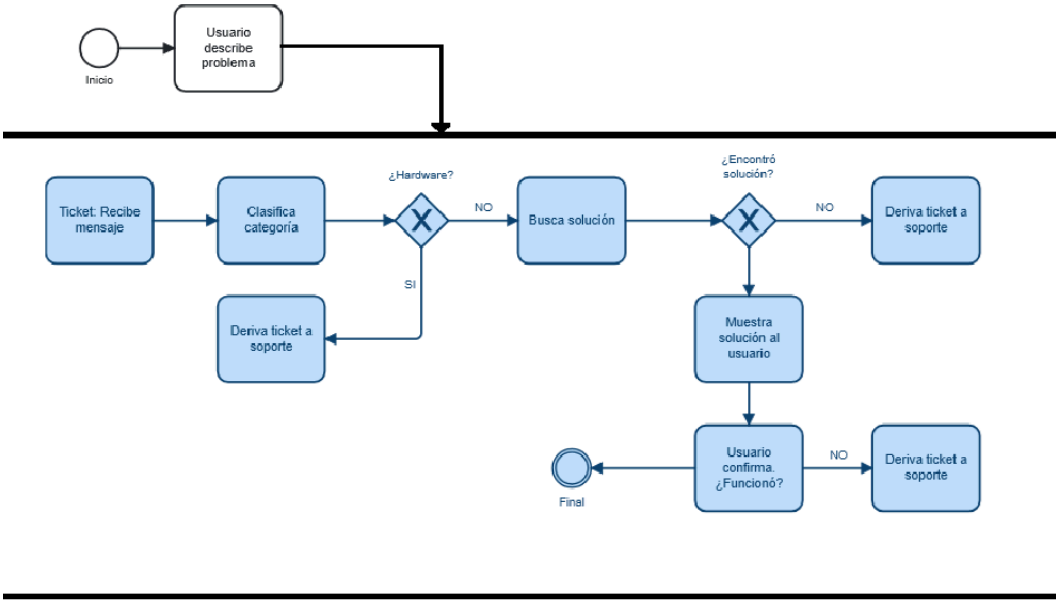
Para la persistencia de información, se implementaron dos archivos JSON:

- **base_conocimiento.json**: Contiene las soluciones predefinidas para problemas clasificados por categoría. Cada entrada incluye una palabra clave y la solución correspondiente.
- **tickets.json**: Registra cada solicitud generada, almacenando el identificador, el usuario, la categoría, la descripción y el estado del ticket.

2.2.4 Lógica de Decisiones

La lógica de decisiones se implementa mediante estructuras condicionales (if/else) que reflejan los puntos de decisión definidos en el diagrama BPMN. Los principales criterios de decisión son:

1. **Clasificación de categoría**: Se identifican palabras clave en la descripción del usuario. Si contiene "impresora" o "hardware" → categoría = hardware. Si contiene "contraseña" o "acceso" → categoría = acceso. En caso contrario → categoría = software.
2. **Disponibilidad de solución**: Se consulta la base de conocimiento buscando coincidencias entre las palabras clave y la descripción del problema.
3. **Confirmación del usuario**: Se evalúa si la solución sugerida fue efectiva según la respuesta del usuario.



(Imagen del Diagrama BPMN del proyecto)

2.3 Documentación y Calidad

2.3.1 Diccionario de Datos

Entidad	Atributo	Tipo	Descripción
Ticket	id	int	Identificador único del ticket.
Ticket	usuario	string	Nombre del usuario que reporta el problema.
Ticket	categoria	string	Categoría del problema (software/hardware/acceso).
Ticket	descripcion	string	Descripción detallada del problema.
Ticket	estado	string	Estado del ticket (abierto/cerrado/derivado).
Ticket	solucion	string	Solución aplicada (si corresponde).
BaseConocimiento	categoria	string	Categoría a la que pertenece la solución.
BaseConocimiento	palabra_clave	string	Término que activa la solución.
BaseConocimiento	solucion	string	Descripción de la solución sugerida.

2.3.2 Manual de Usuario

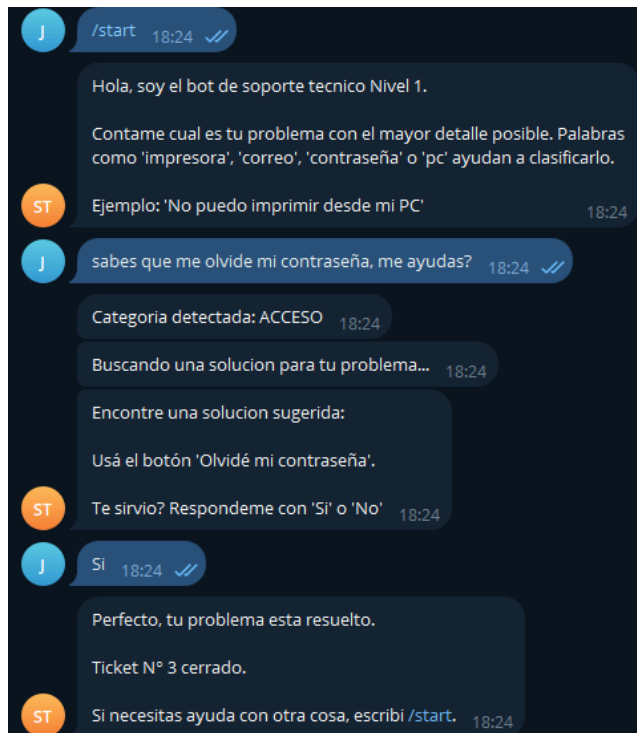
El chatbot funciona a través de la plataforma Telegram. Para iniciar una solicitud, el usuario debe enviar el comando /start. A partir de ese momento, el bot guiará al usuario mediante mensajes interactivos.

Comandos disponibles:

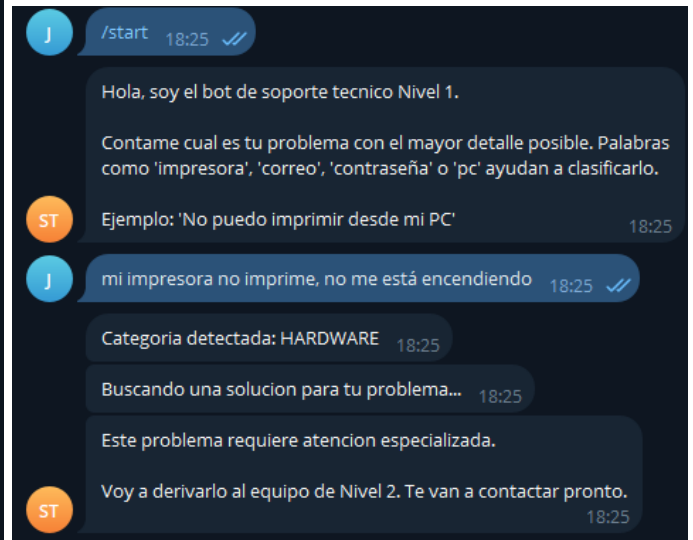
- /start: Inicia el proceso de solicitud de soporte.
- /cancelar: Cancela la solicitud en curso.
- /estado: Consulta el estado del último ticket generado (opcional).

Flujo de interacción típico:

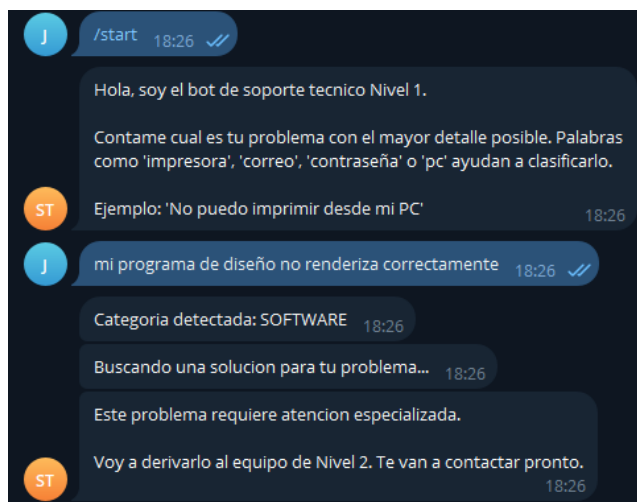
1. El usuario envía /start.
2. El bot responde: "👋 Hola, soy el bot de soporte técnico. Describí tu problema."
3. El usuario describe su problema (ej: "No puedo imprimir").
4. El bot clasifica el problema y responde con una solución o deriva a Nivel 2.
5. Si se ofrece solución, el bot pregunta: "¿Te funcionó? (Sí/No)".
6. El usuario responde "Sí" o "No" según corresponda.
7. El bot confirma el cierre del ticket o deriva el caso.

Manejo de Errores - BOT

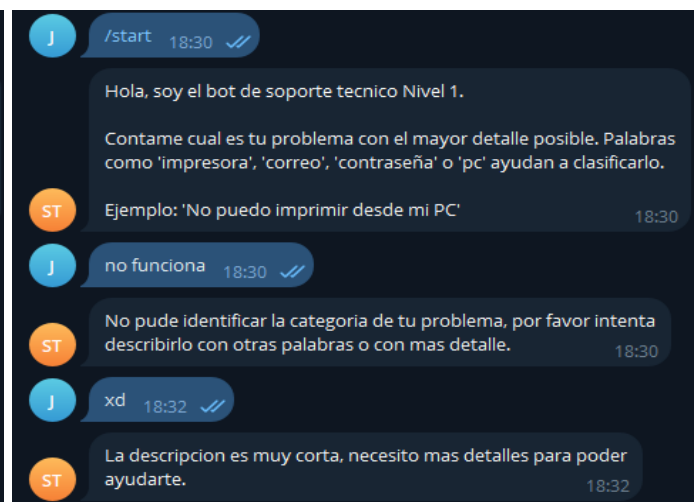
(Se observa interacción BOT con error CONTRASEÑA)



(Se observa interacción por HW, se deriva directamente)



(Se observa interacción por SW, deriva a AGENTE)



(Se observa interacción incorrecta de parte del usuario, no se deriva)

Interacciones con IAs para desarrollo proyecto:

¿Cómo estructuro un proyecto de chatbot en Python con telegram bot?

Estructurar bien tu proyecto desde el primer día te va a ahorrar muchísimos dolores de cabeza (especialmente cuando quieras agregar nuevos comandos o mantener tus credenciales a salvo).

Para un bot de Telegram en Python, la librería más popular y robusta es `python-telegram-bot`. Aquí tienes la estructura de archivos recomendada (estilo producción) y el paso a paso para armarlo.

1. Estructura de Directorios Recomendada

Para mantener el código ordenado, lo ideal es separar la lógica de los comandos, la configuración y el punto de entrada del bot.

Plaintext

```
mi_telegram_bot/  
├── config/  
│   ├── __init__.py      # Configuración del sistema  
│   └── settings.py      # Carga de variables de entorno  
├── handlers/  
│   ├── __init__.py      # Lógica de los comandos y mensajes  
│   ├── start.py         # Comando /start y /help  
│   └── messages.py      # Procesamiento de texto o IA  
├── utils/  
│   ├── __init__.py      # Funciones de ayuda (APIs externas, formato, etc.)  
│   └── api_client.py  
├── .env                 # Variables secretas (NO subir a GitHub)  
├── .gitignore           # Archivos a ignorar por Git  
├── README.md            # Documentación del proyecto  
├── requirements.txt     # Dependencias del proyecto  
└── main.py              # Archivo principal (Punto de entrada)
```

¿que debo implementar en el comienzo de la creación de un chatbot de telegram, podrías dejarme algun link a la biblioteca del mismo?

Q. Read 9 web pages

Para empezar con un chatbot de Telegram, debes implementar estos pasos básicos:

1. Crear el bot con BotFather en Telegram y guardar el token que te da
2. Instalar la biblioteca con `pip install python-telegram-bot`
3. Escribir el código base: importar la librería, construir la aplicación con tu token y definir un manejador para el comando `/start`
4. Iniciar el bot con `app.run_polling()` para que empiece a escuchar mensajes

Aquí tienes el enlace a la documentación oficial: `python-telegram-bot.readthedocs.io`

Nota: La biblioteca `python-telegram-bot` es la más completa y profesional, pero requiere manejo de programación asíncrona (`async/await`). Si buscas algo más sencillo para empezar, existe `pyTelegramBotAPI` (también llamada Telebot).

¿como implemento una máquina de estados en un chatbot de soporte técnico?

Q. Read 10 web pages

Para implementar una máquina de estados en tu bot de soporte, puedes usar la clase `ConversationHandler` de la biblioteca `python-telegram-bot`. Te explico los pasos clave:

- **Define los puntos de entrada (`entry_points`)**: Son los comandos o mensajes que inician la conversación, como `/start` o "Quiero ayuda". Cuando un usuario los usa, se entra al flujo de la máquina de estados.
- **Crea los estados (`states`)**: Aquí defines cada paso del diálogo. Por ejemplo, un estado `SELECCIONAR_PROBLEMA` donde el bot pregunta qué tipo de problema tiene el usuario, y un estado `PEDIR_DETALLES` donde pide más información. A cada estado le asignas un `MessageHandler` para procesar lo que el usuario responda en ese momento.
- **Configura las salidas de emergencia (`fallbacks`)**: Es una lista de handlers para manejar imprevistos, como un comando `/cancel` para que el usuario pueda salir del flujo en cualquier momento, o una respuesta genérica cuando el usuario escribe algo que no esperas en un estado específico.
- **Controla el flujo**: Dentro de la función que maneja un estado, devuelves el nombre del siguiente estado para avanzar, o devuelves `END` o `-1` para finalizar la conversación. Si no devuelves nada (`None`), el estado no cambia.

Un detalle importante: esta implementación está pensada para procesar las actualizaciones una por una. Por eso, si usas `ConversationHandler`, asegurate de desactivar las actualizaciones concurrentes en la configuración de tu aplicación (`ApplicationBuilder.concurrent_updates = False`).

Puedes ver todos los detalles en la documentación oficial: `ConversationHandler — python-telegram-bot Documentation`.

¿Qué estados necesito para un bot de soporte técnico Nivel 1?

Q. Read 10 web pages

Para un bot de soporte de Nivel 1, puedes usar el `ConversationHandler` con estos 4 estados básicos:

- `SELECCIONAR_PROBLEMA`: Punto de entrada. Muestra un menú con categorías (ej. "Problemas de cuenta", "Asistencia técnica").
- `RECOGER_DETALLES`: Pide información específica sobre el problema seleccionado.
- `CONFIRMAR`: Muestra un resumen de lo que entendió el bot y da la opción de confirmar o corregir.
- `END`: Finaliza la conversación y deriva a un agente humano o da una respuesta de cierre.

Aquí tienes un ejemplo de cómo estructurarías esos estados con la lógica de `ConversationHandler`:

- `entry_points`: Iniciaría con un comando como `/soporte`.
- `states`: Definirías cada estado con su manejador.
 - `SELECCIONAR_PROBLEMA`: `MessageHandler` para capturar la categoría elegida.
 - `RECOGER_DETALLES`: `MessageHandler` para el texto de descripción del problema.
 - `CONFIRMAR`: `CallbackQueryHandler` para los botones de "Si/No".
- `fallbacks`: Añadir un manejador para `/cancelar` para que el usuario pueda salir del flujo en cualquier momento.

Este diseño te permite un flujo guiado, manejando cada tipo de respuesta que da el usuario en el momento adecuado.

Conclusión

El desarrollo de este trabajo me permitió aplicar los conceptos de modelado de procesos y programación en un caso concreto de soporte técnico. La implementación del chatbot demostró que es posible automatizar la clasificación y resolución de incidencias, reduciendo tiempos de respuesta y liberando al equipo de tareas repetitivas.

El uso de BPMN 2.0 resultó clave para comprender el flujo antes de programar, asegurando coherencia entre el diseño y el código. La máquina de estados permite gestionar conversaciones de manera ordenada, mientras que la base de datos simulada evidenció la importancia de la persistencia.

Los mecanismos de manejo de errores incorporados garantizan robustez ante entradas incorrectas, reflejando la necesidad de considerar todos los escenarios posibles en una solución automatizada.

En definitiva, el trabajo reafirma la importancia de integrar conocimientos de negocio y tecnología para aportar soluciones con valor real a las organizaciones.

También se me hace importante recalcar el uso de la IA en el trabajo, ya que sin la misma, muchos conceptos de la biblioteca *“from telegram import Update //from telegram.ext import Application, CommandHandler, MessageHandler, filters, ConversationHandler, ContextTypes”* no los hubiera entendido, y necesité de la misma + repositorios como para poder cumplir con la función del BOT

Referencias

Python Software Foundation. (2026). Python Documentation. Recuperado de <https://docs.python.org/>

python-telegram-bot. (2026). Documentation. Recuperado de <https://docs.python-telegram-bot.org/>

Tecnicatura Universitaria en Programación. Organización Empresarial. Unidad (2026). (1° ed.).

Universidad Tecnológica Nacional.